HOSTFALL — ANDROID ADDENDUM

This document is a supplement to `HOSTFALL_DESIGN.md`. It covers ONLY the changes needed to ship an Android build alongside the desktop HTML version. The core game design — enemy stats, boss mechanics, weapon balance, screen layouts — is **unchanged**. This is purely an input-layer and packaging addition.

The desktop keyboard build is the source of truth. The Android build wraps the same `index.html` and adds a touch input layer.

---

1. DESIGN PHILOSOPHY (READ THIS FIRST)

Hostfall is intentionally Atari-era in its mechanics, not just its visuals. The hero has a **persistent facing direction** and can only shoot or throw grenades in that direction. There is **no auto-aim, no twin-stick, no strafe-shooting**. This is a deliberate constraint, identical in spirit to Berzerk (1980).

Do not "compensate" for facing-locked combat by making the game easier. Do not lower enemy counts, slow enemy bullets, extend i-frames, or stretch boss telegraph windows. The constraint *is* the challenge. The player learning to position before committing to fire is the entire skill loop.

If a playtest says "this is hard," that is the correct outcome. Berzerk was hard. Build the game as specified.

---

2. INPUT MODEL

Facing direction

The hero has a `facing` property — one of 8 compass directions (N, NE, E, SE, S, SW, W, NW). The hero's sprite is drawn rotated/flipped to match facing. A small visible indicator (gun barrel, body angle) makes facing readable at a glance.

Facing updates whenever the hero moves. The direction the hero is currently moving = the direction they are facing. When they stop moving, facing persists from the last movement vector.

Desktop (unchanged from main spec)

- WASD = move
- Arrow keys = also move AND set facing in that arrow's direction (this is the desktop equivalent of "pivot")
- Holding an arrow key while moving with WASD = override facing to the arrow direction *only while held*
- Space = grenade
- Shift = sprint
- E = interact
- Tab = cycle weapon
- R = reload
- Escape = pause

(Note: this is a small revision from the original spec, which was twin-stick. The new model is facing-locked on desktop too, for consistency. Arrow keys now set facing rather than firing. **Firing is on a dedicated key — assign Z or Ctrl as the FIRE key.** Hold to fire continuously.)

Android (touch)

- **Tap on game world** = path the hero to that location. Hero walks there, facing updates to current movement direction along the path.
- **Tap within 32px of hero** = pivot only. Hero turns to face the tap direction without moving. (Critical escape valve so the player isn't forced to walk into walls to reorient.)
- **FIRE button** (large, bottom-right) = hold to fire continuously in current facing direction at the weapon's normal fire rate.
- **GRENADE button** (above FIRE) = single tap throws a grenade in facing direction. Greyed out when grenade count is 0.
- **WEAPON button** (bottom-left, smaller) = cycles to next owned weapon. Shows brief weapon name popup on tap.
- **PAUSE icon** (top-right) = pauses game.
- **Sprint** = double-tap on game world to start a sprint-pathed move (uses stamina as desktop). Releases when stamina runs out or destination reached.

Tap behavior details

- Tapping a new destination while the hero is moving = immediately reroute. No commitment.
- Tapping near a screen edge with an open exit = walk to the edge; the last 16px of any open edge auto-triggers room transition.
- Tapping a wall = path to the closest reachable tile adjacent to the wall.

---

3. PATHFINDING

Implement A* on a 16×16-pixel tile grid. Each room is 20×15 tiles (320×240 / 16). Walls are blocked tiles; everything else is walkable.

- Recompute path only on new tap or when blocked mid-path (e.g. a new wall appears, which shouldn't happen in v1 but be defensive).
- Hero follows the path tile-by-tile. Movement speed unchanged from desktop (90 px/s base, 135 sprint).
- Facing updates each tile transition based on the vector to the next tile.
- Hero collision (8×8 hitbox) still applies — pathfinder treats the hero as occupying its tile, with sub-tile movement smoothed.
- If pathfinding fails (no route), hero stays put. No error, no UI feedback — just doesn't move.

Cost is negligible: A* on a 300-tile grid completes in under a millisecond. Don't overengineer.

---

4. TOUCH UI LAYOUT

```
┌──────────────────────────────────────┐
│ ♥♥♥♥♥    PULSE  AMMO ∞        [ ▮▮ ]  │  HUD top bar (24px tall)
│                                      │
│                                      │
│        [game world — tap to move]    │
│                                      │
│                                      │
│                              ┌─────┐ │
│                              │ GRN │ │  56×56
│                              │  3  │ │
│                              └─────┘ │
│  ┌────┐                      ┌────┐  │
│  │ WP │                      │FIRE│  │  72×72
│  └────┘                      └────┘  │
└──────────────────────────────────────┘
```

- HUD overlays in HTML divs, positioned absolutely over the canvas.
- Buttons use `pointer-events: auto`; the game-world tap zone uses `pointer-events: none` and the canvas handles raw taps.
- Buttons are semi-transparent (60% opacity at rest, 90% on press) so they don't fully obscure gameplay.
- All button hit areas at least 64×64px minimum even if visual is smaller — fat-finger margin.
- FIRE button is the largest because it's pressed most.
- Color-code: FIRE red, GRENADE orange, WEAPON white, PAUSE white. Matches existing palette.

---

5. ANDROID-SPECIFIC TECHNICAL REQUIREMENTS

1. **Audio unlock.** Web Audio on Android requires a user gesture before `AudioContext` will play. On first touch anywhere, call `audioContext.resume()`. Wrap this in a one-shot listener.
2. **Orientation lock.** Lock to landscape in Capacitor config (`capacitor.config.json`):

```json
"plugins": {
  "ScreenOrientation": { "orientation": "landscape" }
}
```

Also set in `AndroidManifest.xml`: `android:screenOrientation="sensorLandscape"`.

3. **Back button.** Map Android's hardware back button to pause. Use Capacitor's `App.addListener('backButton', ...)`.
4. **Status bar.** Hide it. Fullscreen immersive mode. Configure in Capacitor.
5. **Sleep prevention.** Use Capacitor's `KeepAwake` plugin to prevent screen sleep during gameplay. Release when paused.
6. **Viewport meta.** Ensure `<meta name="viewport" content="width=device-width, initial-scale=1.0, maximum-scale=1.0, user-scalable=no, viewport-fit=cover">` is in the HTML head. Prevents pinch-zoom and adjusts for notched displays.
7. **Canvas scaling.** Existing CSS scaling already handles arbitrary screen sizes — confirm `image-rendering: pixelated` works on Android WebView (it does, Chromium-based).

---

6. PACKAGING WITH CAPACITOR

Build steps for Claude Code:

```bash
# From project root, after index.html is built and tested
npm init -y
npm install @capacitor/core @capacitor/cli
npx cap init "Hostfall" "com.yourname.hostfall" --web-dir="."
npm install @capacitor/android
npx cap add android
npx cap sync android
npx cap open android
```

Then in Android Studio: Build → Build APK(s) → debug APK output to `android/app/build/outputs/apk/debug/app-debug.apk`.

For sideloading to a personal device, debug APK is sufficient. For Play Store distribution, generate a release keystore and signed AAB — out of scope for v1.

Capacitor config (`capacitor.config.json`)

```json
{
  "appId": "com.yourname.hostfall",
  "appName": "Hostfall",
  "webDir": ".",
  "plugins": {
    "ScreenOrientation": {
      "orientation": "landscape"
    },
    "StatusBar": {
      "style": "DARK",
      "overlaysWebView": false
    },
    "KeepAwake": {}
  },
  "android": {
    "backgroundColor": "#000000"
  }
}
```

---

7. INPUT MODE DETECTION

The same `index.html` should run on desktop and Android, with the right UI shown automatically.

- On load, hide touch UI by default.
- Listen for first input event:
  - First event is `keydown` → desktop mode, leave touch UI hidden.
  - First event is `touchstart` → Android mode, show touch UI overlay.
- Allow live switching: if a Bluetooth keyboard is connected to a tablet later in the session and a key is pressed, hide touch UI; if a touch happens after that, show it again.

This lets a single build serve both platforms cleanly. No conditional builds, no separate code paths in game logic.

---

8. BUILD ORDER FOR ANDROID PHASE

After the desktop HTML build is complete and tested:

1. **Refactor input layer.** Make sure all game logic reads from a unified `input` state object (move vector, facing override, fire held, grenade pressed, weapon-cycle pressed, pause). Keyboard handlers write to this object. This may already be done; confirm.
2. **Add touch UI overlay.** HTML divs, CSS positioning, no game logic yet.
3. **Wire FIRE/GRENADE/WEAPON/PAUSE buttons** to write to the same input state object. Test in browser using mobile device emulation in Chrome DevTools.
4. **Add tap-to-move handler** with A* pathfinding. Hero walks paths, facing updates.
5. **Add tap-near-hero pivot logic.**
6. **Add input mode auto-detection** (keyboard vs touch).
7. **Test all four acts on a touch device** (Chrome DevTools mobile emulation, then a real Android phone via USB debugging).
8. **Set up Capacitor.** `npm init`, install Capacitor, add Android platform.
9. **Configure Capacitor:** orientation lock, audio unlock, back button mapping, status bar, keep-awake.
10. **Build debug APK.** Install on device. Playtest.
11. **Polish.** Adjust button sizes/positions based on real-thumb feel. Tune transparency. Consider haptic feedback on FIRE (Capacitor's Haptics plugin) — short tick on each shot, slightly stronger on hits taken. Optional but adds a lot.

---

9. WHAT IS EXPLICITLY OUT OF SCOPE FOR ANDROID v1

- Auto-aim
- Twin-stick virtual joysticks
- Tap-an-enemy-to-target shortcuts
- Difficulty options
- Cloud save / Google Play Games integration
- IAP, ads, analytics
- Tablet-specific layouts (it'll work on tablets, it just won't have a special tablet UI)
- iOS port (Capacitor supports it, but not in v1)
- Play Store release (debug-signed APK only)

END OF ADDENDUM

The desktop HTML game is the foundation. This addendum adds touch input and Android packaging without touching gameplay numbers. Ship the desktop build first, then layer this on as a clean second pass.